

Sensor Fusion/Navigation

Autonomous motion on our iHerb robots work due to several ROS 2 systems connected together in a feedback pipeline. The robot first estimates where it is, Nav2 decides how it should move to follow a saved route, and the motor control system converts Nav2's movement commands into the joystick commands that already drive the robot manually.

This autonomous motion can also have crash avoidance and scene segmentation added in order to improve behavior and help the robot react more dynamically.

The system can be understood as **five** major parts:

1. Localization and sensor fusion

2. Route recording and route following

3. Nav2 path planning and control

4. Autonomous command conversion through the joystick pipeline

5. Motor command execution and odometry feedback

1. Localization: Knowing Where the Robot Is

The robot **must** know its position and heading. We are using **sensor fusion** to estimate the robot's pose. This is primarily focused in our **brex_localization** node in our ROS2 workspace.

A key detail is that the Xsens sensor already has onboard filtering internally. The MTi-680G has its own **onboard Kalman filter**, which **fuses** its internal **inertial/GNSS** measurements before **publishing** data to ROS.

TLDR: it does sensor fusion already. But the reason we have a EKF in our system is because we want to add more sensors in the future. SO, this is a cascaded EKF system. Adjust the values if necessary.

That filtered Xsens output is then used by the ROS robot_localization EKF.

Flow of the system is:

Xsens onboard extended Kalman filter

- > ROS sensor topics such as `/imu/data`, `/gnss`, and `/odometry` (encoder data coming serially)
- > robot_localization EKF
- > `/odometry/filtered`

Note that the choice of using an Extended Kalman Filter (EKF) is because we need to correct the inherent errors (noise and drift) of individual sensors, ensuring the most accurate pose

estimation. The EKF functions as a statistically optimal state estimator by utilizing a two-step process: a **prediction** based on the robot's motion, and a **correction/update** step where actual sensor measurements (odometry(encoder data), IMU, and GNSS) are fused, giving us a probability of where the robot is.

It assigns statistical weight (covariance) to each sensor measurement based on its estimated reliability, effectively minimizing the uncertainty across all inputs to produce a single, highly stable, and accurate estimate on the `/odometry/filtered` topic.

In short, we use the Xsens onboard filter as a pre-filtered sensor source, and then use `robot_localization` as the robot-level EKF for Nav2 to use.

The **main** sensor inputs are:

- Wheel/platform odometry from the motor system
- Filtered IMU orientation and angular velocity data from the Xsens sensor
- GPS/GNSS data from the Xsens sensor
- GPS-derived local odometry from `navsat_transform_node` when GPS is enabled

The Xsens driver publishes sensor data such as `/imu/data` and `/gnss`. The motor control system publishes wheel odometry, such as `/wheel/odom` or `/odometry`, depending on the launch/remapping being used. The `robot_localization` EKF combines these inputs and produces the filtered localization output.

The most important localization output is:

`/odometry/filtered`

This topic contains the robot's best estimate of its current position, velocity, and orientation. It is the pose estimate that Nav2 uses to understand where the robot is while it is navigating.

The EKF also publishes the TF transform between:

`odom -> base_link`

This transform tells ROS where the robot body, `base_link`, is located inside the odometry frame, `odom`. Nav2 depends on this transform. If this transform is missing, Nav2 cannot reliably plan or control motion because it does not know the robot's current pose.

For outdoor GPS-based work, `navsat_transform_node` is also used. Its job is to convert latitude and longitude from GPS into a local ROS coordinate frame. GPS gives global coordinates, but Nav2 needs local map-style coordinates such as `x` and `y`. The `navsat_transform_node` bridges that gap. It can also provide the `/fromLL` service, which converts latitude/longitude goals into map-frame points that Nav2 can use.

One important caution is that the Xsens output is already filtered. If the ROS EKF fuses several Xsens-derived topics that contain overlapping information, such as Xsens odometry, Xsens IMU orientation, and GPS-derived odometry, the EKF may be using correlated measurements as if they were independent. That does not mean the system is wrong, but it is something to keep in mind when tuning covariances or debugging GPS jumps.

Localization does not directly drive the motors. Localization provides position information. Nav2 uses that position information to decide what velocity commands should be sent.

In short, BREx_Localization is just giving us the robot's position data, and Nav2 uses the position info to drive velocity commands (this is when it is autonomous).

2. Route Recording: Saving a Path for the Robot to Replay

Route replaying begins by manually driving the robot while localization is working. During this manual drive, the route recorder watches the robot's pose in the map frame, usually through:

`map -> base_link`

The route recorder samples the robot's position and heading as the robot moves. It does not save every tiny movement. Instead, it saves a new waypoint only after the robot has moved far enough, turned enough, or enough time has passed. This prevents the route file from becoming too dense.

The recorded route is saved as a YAML file, usually:

`/home/brex/outdoor_route.yaml`

The route file contains waypoints like this:

```
waypoints:
- x: -1.0
  y: 0.0
  yaw: 0.0
  frame_id: map
- x: -2.0
  y: 0.0
  yaw: 0.0
  frame_id: map
```

Each waypoint represents a position and heading that the robot should later drive through autonomously. The x and y values are positions in the map frame, and **yaw** is the robot's heading at that point.

The main file to look at is [outdoor_saved_costmap_route.launch.py](#)

This is located at [ros2_ws/src/brex_nav2/brex_nav2](#) .

3. Route Following: Sending the Saved Route to Nav2

When replaying the route, the route follower loads the saved YAML route file and **turns each waypoint** into a **ROS PoseStamped message**. These poses are then sent to **Nav2** using the **NavigateThroughPoses** action.

The important Nav2 action is:

[navigate_through_poses](#)

This action tells Nav2 to drive through a list of poses in order.

If the waypoints are already saved as **map-frame x/y** points, the route follower sends them directly to Nav2. If the waypoints are saved as **GPS latitude/longitude** points, the route follower first calls the [/fromLL](#) service from [robot_localization](#) to convert the GPS coordinates into map coordinates.

The route follower does not directly command the wheels. Its job is only to give Nav2 the route. Nav2 then handles the actual navigation behavior.

4. Nav2: Planning and Producing Velocity Commands

Nav2 is the navigation system. Once it receives a route, it continuously compares:

- **Where** the robot **currently** is, from [/odometry/filtered](#) and *TF*
- **Where** the robot is **supposed to go**, from the **route waypoints**

Nav2 uses its planner and controller servers to **generate** movement commands. The planner decides the path through the **map** or **costmap**. The controller decides what the robot should do right now to follow that path.

The output from Nav2 is a velocity command on:

[/cmd_vel](#)

This message is a geometry_msgs/Twist. It contains:

linear.x -> forward/backward speed
angular.z -> turning speed

For a normal differential-style ROS robot, /cmd_vel might go directly to a motor driver. On BREx, the existing motor system was originally built around joystick control. Because of that, the autonomous /cmd_vel command is converted into a virtual joystick command that then goes into a microcontroller that then communicates with a motor driver. This then commands the motors.

This is one of the most important parts of how autonomy was integrated without replacing the original manual driving system.

5. Auto Mode: Converting Nav2 Commands Into Joystick Commands

In auto mode, Nav2 publishes /cmd_vel. That command flows through the autonomous control path.

The command path is approximately:

Nav2
-> /cmd_vel
-> auto_cmd_mux
-> /cmd_vel_selected
-> virtual_joy
-> /joy_auto
-> joy_mux
-> /joy
-> serial_cmd_sender
-> motor controllers

The auto_cmd_mux chooses which autonomous velocity command should be used. If collision avoidance is disabled, it passes the raw Nav2 /cmd_vel. If collision avoidance is enabled, it can instead use a safer command from /cmd_vel_safe.

The virtual_joy node converts the selected Twist command into a sensor_msgs/Joy message. This makes the autonomous command look like joystick input. For example, forward velocity becomes a joystick forward axis value, and turning velocity becomes a joystick turn axis value.

The joy_mux node chooses whether the robot should listen to the real controller or the autonomous virtual controller. It has two modes:

manual
auto

In manual mode, it passes the real joystick topic through. In auto mode, it passes the virtual joystick topic through.

This is how the same robot can be driven manually or autonomously without needing two completely separate motor systems.

6. Motor Execution: Sending Commands to the Controllers

After joy_mux publishes the selected joystick message on /joy, the serial_cmd_sender reads it and converts it into motor commands.

The motor command sender calculates left and right wheel speeds from the joystick axes. It then sends serial commands to the left and right motor controllers through ports such as:

```
/dev/ttyACM0  
/dev/ttyACM1
```

The motor controllers move the robot. As the robot moves, wheel feedback is read back by the serial node. That feedback is used to calculate wheel odometry, which is published back into ROS. This closes the loop: movement produces odometry, odometry updates localization, localization updates Nav2, and Nav2 updates the next motion command.

7. The Full Autonomous Loop

The full route replay loop works like this:

Xsens onboard Kalman filter

- > Xsens ROS topics such as /imu/data and /gnss
- > wheel/platform odometry
- > robot_localization EKF
- > /odometry/filtered and TF
- > Nav2 knows the robot pose
- > route follower sends saved waypoints to Nav2
- > Nav2 plans and controls motion

- > Nav2 publishes /cmd_vel
- > virtual_joy converts /cmd_vel into joystick-style input
- > joy_mux selects auto mode
- > serial_cmd_sender sends motor commands
- > robot moves
- > wheel feedback updates odometry
- > localization updates again

This loop repeats many times per second while the robot is driving autonomously.

8. Why Route Replaying Works

Route replaying works because the saved route and the live robot pose are in the same coordinate system. When the robot is manually driven during recording, the recorder saves where the robot went. Later, during replay, Nav2 receives those saved poses and tries to drive the robot through them.

The EKF keeps estimating the robot's current pose, and Nav2 continuously corrects the robot's motion based on the difference between the current pose and the next target pose. If the robot drifts slightly, Nav2 adjusts the velocity command to bring it back toward the route.

The joystick system is still involved because it is the final control interface to the motor controllers. In manual mode, real joystick input drives the robot. In auto mode, Nav2's velocity commands are converted into virtual joystick input, allowing the same motor pipeline to be reused.

Conclusion

Autonomous route replay on BREx is not one single node. It is a complete ROS pipeline. The Xsens sensor first performs onboard filtering, robot_localization then produces the robot-level filtered odometry estimate, the route follower gives Nav2 a list of saved waypoints, Nav2 calculates velocity commands, and the joystick/motor system converts those commands into serial motor commands. The robot's movement then feeds this back into localization, creating a continuous feedback loop that allows the robot to replay a route autonomously.